

Lab-6.3

Task Description#1 (Classes)

- Use AI to complete a Student class with attributes and a method.
- Check output
- Analyze the code generated by AI tool

Instructions:

- **Initialize class with attributes like name, roll no, marks**
- **Method to display student details**
- **Method to calculate grade based on marks (A: >=90, B: >=75, C: >=60, else Fail)**

Start Writing code and auto complete using any AI tool

Expected Output#1

- Class with constructor and display_details() method

```

def calculate_marks(marks):
    if marks >= 90:
        return "A"
    elif marks >= 75:
        return "B"
    elif marks >= 60:
        return "C"
    else:
        return "F"

def student_marks(name, marks, roll_no):
    print(f"Name: {name}, Marks: {marks}, Roll No: {roll_no}")
    print(f"Garde:", calculate_marks(marks))
    print("-----")

# INSERT_YOUR_CODE

def main():
    n = int(input("Enter number of students: "))
    for i in range(n):
        name = input("Enter student name: ")
        roll_no = input("Enter roll number: ")
        while True:
            try:
                marks = float(input("Enter marks: "))
                break
            except ValueError:
                print("Please enter a valid number for marks.")
        student_marks(name, marks, roll_no)

if __name__ == "__main__":
    main()

```

OUTPUT:

```

"c:/Users/Rishitha Reddy/OneDrive/Desktop/AIAC/Lab-6/Task.1.1.py"
Enter number of students: 10
Enter student name: sumalatha
Enter roll number: 3
Enter marks: 90
Name: sumalatha, Marks: 90.0, Roll No: 3
Garde: A
-----

```

Task Description#2 (Loops)

- Prompt AI to complete a function that prints the first 10 multiples of a number using a loop.
- Analyze the generated code
- Ask AI to generate code using other controlled looping

Write code using **For** Loop, later complete code using **While** Loop

Expected Output#2

- Correct loop-based implementation

```
1  def multiple_num(n):
2      print("Multiples of given number using for loop")
3      for i in range(1, 11):
4          print(n * i)
5      print("End of multiples")
6      print("Multiples of given number using while loop")
7      i=1
8      while i <= 10:
9          print(n * i)
10         i += 1
11     print("End of multiples")
12     n=int(input("Enter the number: "))
13     multiple_num(n)
```

OUTPUT:

```
ha Reddy/OneDrive/Desktop/AIAC/Lab-6/Task1.2.py"
Enter the number: 5
Multiples of given number using for loop
5
10
15
20
25
30
35
40
45
50
End of multiples
Multiples of given number using while loop
5
10
15
20
25
30
35
40
45
50
End of multiples
PS C:\Users\Rishitha Reddy>
```

Task Description#3 (Conditional Statements)

- Ask AI to write nested if-elif-else conditionals to classify age groups.
- Analyze the generated code
- Ask AI to generate code using other conditional statements

Table: Age Group Classification Logic

Age Range	Age Group
0 – 12 years	Child
13 – 19 years	Teen
20 – 59 years	Adult
60 years & above	Senior

-

Expected Output#3

- Age classification function with appropriate conditions and with explanation

```

1  def group_age(age):
2      if age < 13:
3          print("child")
4      elif age >= 13 and age <= 19:
5          print("teen")
6      elif age >= 20 and age <= 59:
7          print("adult")
8      else:
9          print("senior citizen")
10
11  # Get input and call the function
12  age = int(input("Enter the age: "))
13  group_age(age)

```

OUTPUT:

```

PS C:\Users\Rishitha Reddy>
Enter the age: 15
teen
PS C:\Users\Rishitha Reddy>

```

Task Description#4 (For and While loops)

- Generate a sum_to_n() function to calculate sum of first n numbers
- Analyze the generated code
- Get suggestions from AI with other controlled looping

Expected Output#4

- Python code with explanation

```
1  def sum_to_n(n):
2      total = 0
3      for i in range(1, n+1):
4          total += i
5      return total
6
7  n = int(input("Enter a number: "))
8  print("Sum of first", n, "numbers is:", sum_to_n(n))
```

OUTPUT:

```
ha Reddy/OneDrive/Desktop/AIAC/Lab-6/T
Enter a number: 4
Sum of first 4 numbers is: 10
PS C:\Users\Rishitha Reddy>
```

Task Description#5 (Class)

- Use AI to build a BankAccount class with deposit, withdraw, and balance methods.
- Analyze the generated code
- Add comments and explain code

Instructions

- Initialize BankAccount class with attributes like name, balance
- Method to deposit amount
- Method to withdraw amount
- Method to check balance

Expected Output#5

- Python code with explanation

```
1 class BankAccount:
2     def __init__(self, name, balance=0):
3         self.name = name
4         self.balance = balance
5
6     def deposit(self, amount):
7         if amount > 0:
8             self.balance += amount
9             print(f"Deposited {amount}. New Balance: {self.balance}")
10        else:
11            print("Deposit amount must be positive.")
12
13    def withdraw(self, amount):
14        if amount > 0:
15            if amount <= self.balance:
16                self.balance -= amount
17                print(f"Withdrew {amount}. Remaining Balance: {self.balance}")
18            else:
19                print("Insufficient balance.")
20        else:
21            print("Withdrawal amount must be positive.")
22
23    def check_balance(self):
24        print(f"Account Holder: {self.name}, Current Balance: {self.balance}")
25
26    account1 = BankAccount("Alice", 1000)
27    account1.check_balance()
28    account1.deposit(500)
29    account1.withdraw(300)
30    account1.withdraw(1500) # Should show insufficient balance
31    account1.check_balance()
```

[Review next file >](#)

OUTPUT:

```
Deposited 500. New Balance: 1500
Withdrew 300. Remaining Balance: 1200
Insufficient balance.
Account Holder: Alice, Current Balance: 1200
PS C:\Users\Rishitha Reddy>
```